Bitcoin Market Sentiment

Prepared by : Ronak Waghmare

Email : ronak818082@gmail.com

Relationship Between Bitcoin Market Sentiment and Trader Performance

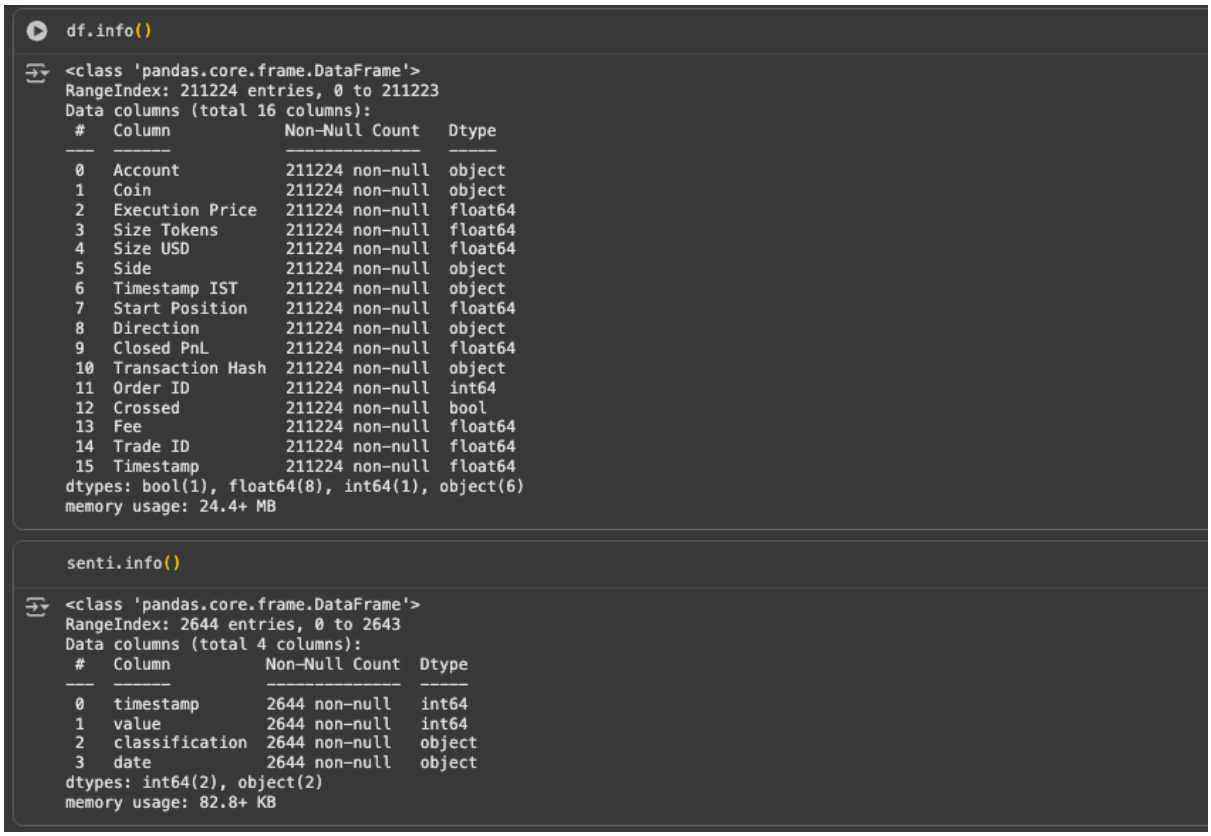
Objective

The goal of this study is to examine how **Bitcoin market sentiment** (Fear vs. Greed) influences **trader behavior and performance**.
We combine two datasets:

- **Fear & Greed Index** (daily classification of market sentiment)
- **Hyperliquid Trader Data** (historical execution-level data for BTC trades)

By merging these, we evaluate:

- How performance varies under Fear vs. Greed
- Behavioral shifts in trade direction, size, and synthetic leverage
- Risk differences (PnL volatility)
- Actionable insights for trading strategies
-



```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211224 entries, 0 to 211223
Data columns (total 16 columns):
 #   Column            Non-Null Count   Dtype  
---  ------            --------------   -----  
 0   Account           211224 non-null  object 
 1   Coin              211224 non-null  object 
 2   Execution Price   211224 non-null  float64
 3   Size Tokens       211224 non-null  float64
 4   Size USD          211224 non-null  float64
 5   Side              211224 non-null  object 
 6   Timestamp IST     211224 non-null  object 
 7   Start Position    211224 non-null  float64
 8   Direction         211224 non-null  object 
 9   Closed PnL        211224 non-null  float64
 10  Transaction Hash  211224 non-null  object 
 11  Order ID          211224 non-null  int64  
 12  Crossed           211224 non-null  bool   
 13  Fee               211224 non-null  float64
 14  Trade ID          211224 non-null  float64
 15  Timestamp         211224 non-null  float64
dtypes: bool(1), float64(8), int64(1), object(6)
memory usage: 24.4+ MB
```

```
senti.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2644 entries, 0 to 2643
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   timestamp       2644 non-null   int64 
 1   value           2644 non-null   int64 
 2   classification  2644 non-null   object
 3   date            2644 non-null   object
dtypes: int64(2), object(2)
memory usage: 82.8+ KB
```

Data Preparation

. Data Sources

- **Fear & Greed Index (fear_greed_index.csv)**
  - Contains daily sentiment classification (Fear / Greed / Neutral).
  - Key fields: date, classification.
- **Historical Trades (historical_data.csv)**
  - Contains transaction-level trade records.
  - Key fields: Timestamp IST, Side, Direction, Closed PnL, Size USD.

a) Date-Time Standardization

- Converted Timestamp IST → Timestamp (datetime64[ns]).
- Extracted **Date** for merging with sentiment data.

```
# Historical trades
df['Timestamp'] = pd.to_datetime(df['Timestamp IST'], format='%d/%m/%y %H:%M', errors='coerce')
df['Date'] = df['Timestamp'].dt.date   # <-- this creates a clean date column
```

```
senti['Date'] = pd.to_datetime(senti['date'], format='%d/%m/%y', errors='coerce').dt.date
```

b) Merging Datasets

- Merged **trades** with **sentiment classification** on Date (left join).
- Dropped irrelevant columns: Transaction Hash, Order ID, Trade ID, Account.

c) Feature Engineering

Encoding Trade Side

- BUY → 1, SELL → 0.

Encoding Trade Direction

- Positive trades (Buy, Open Long, Long > Short, Short > Long) → +1.
- Negative trades (Sell, Open Short) → -1.
- Neutral/technical (Close, Spot Conversion, Auto-Deleveraging) → 0

Profit Flag

- Binary variable: 1 if Closed PnL > 0, else 0

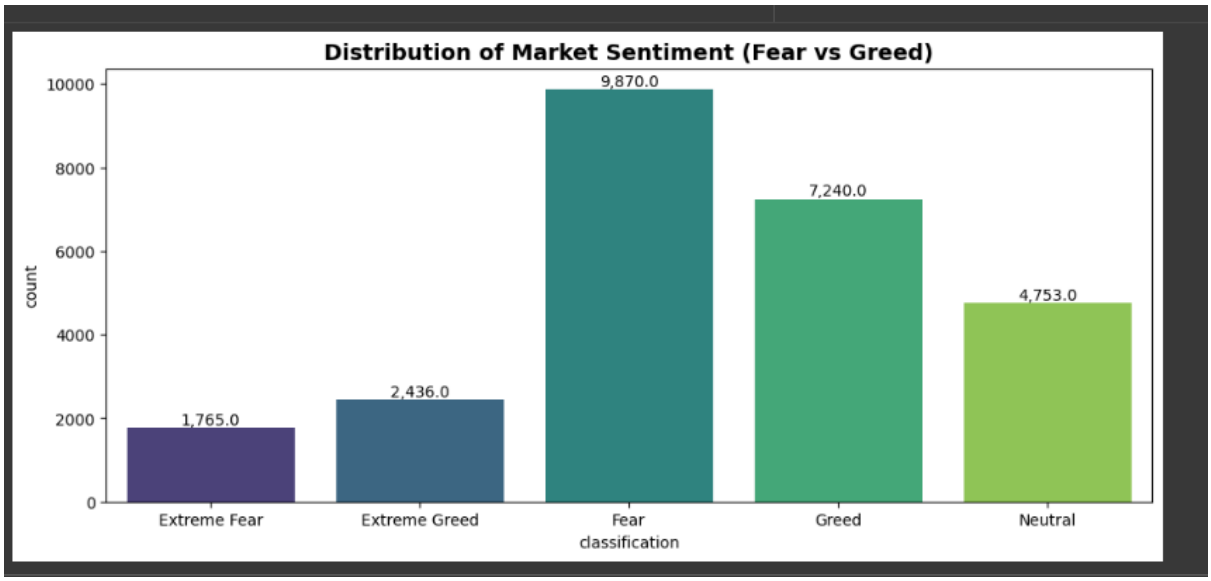
Final Cleanup

  - Dropped unused columns: Direction, Timestamp IST

Exploratory Analysis & Results

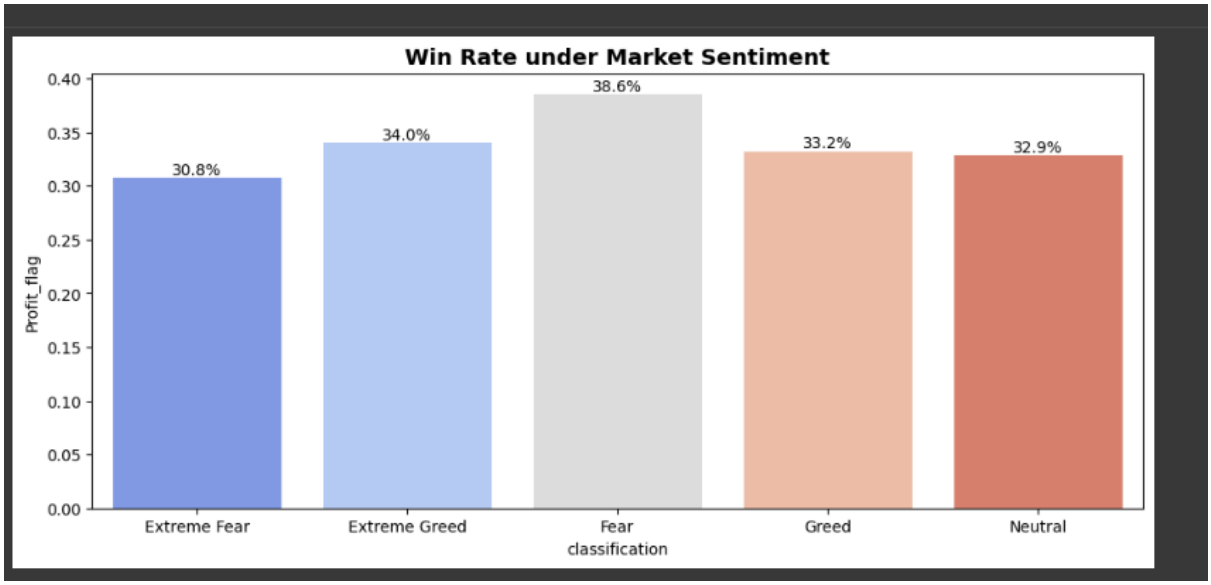
1. Sentiment Distribution

- Count of trades across different market sentiments.
- Visualized with sns.countplot.
- Helps identify if more trades occur during **Fear** vs **Greed** conditions.



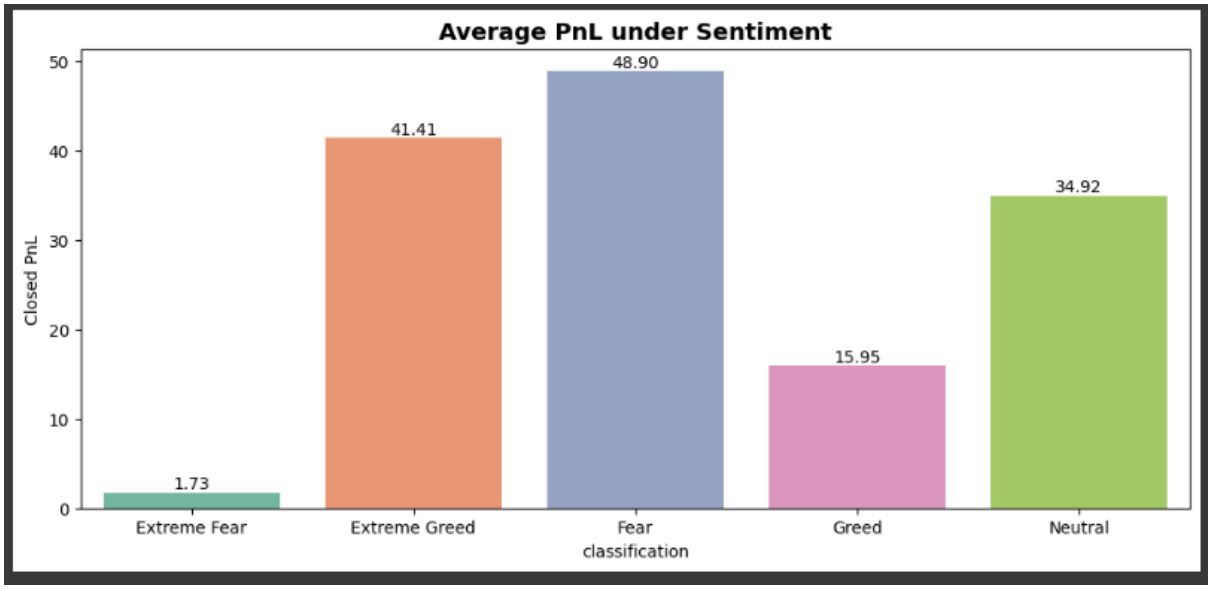
2. Win Rate by Sentiment

- Computed average **Profit_flag** by sentiment.
- Shows how often trades are profitable in **Fear**, **Greed**, or **Neutral** conditions.



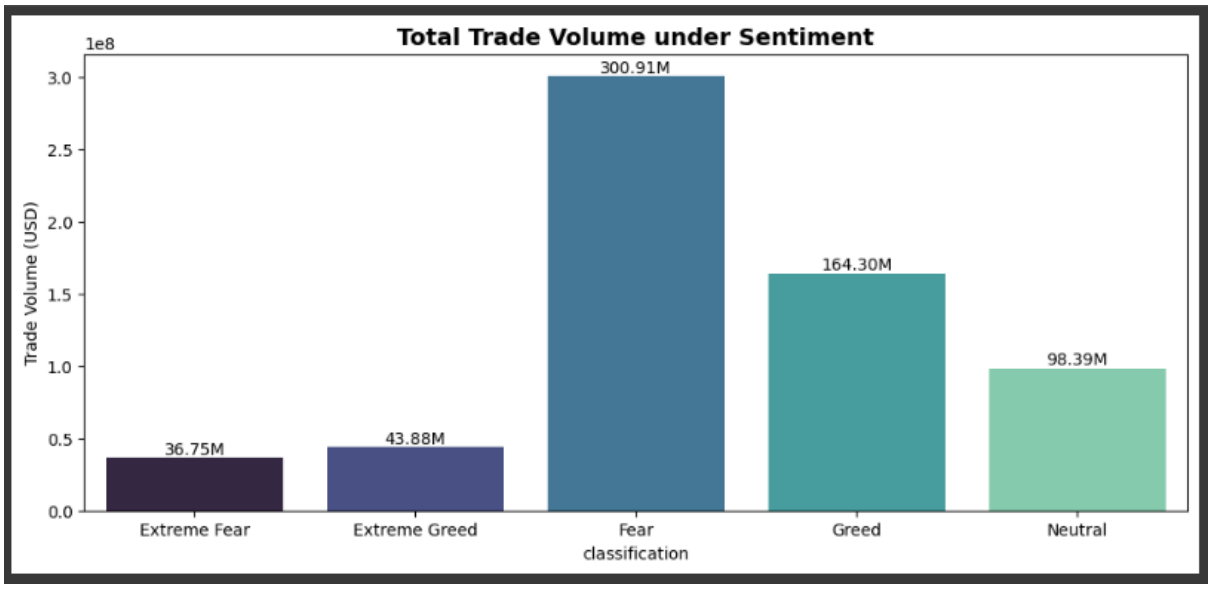
3. Average PnL by Sentiment

- Mean Closed PnL per sentiment group.
- Indicates profitability intensity under different emotional markets.



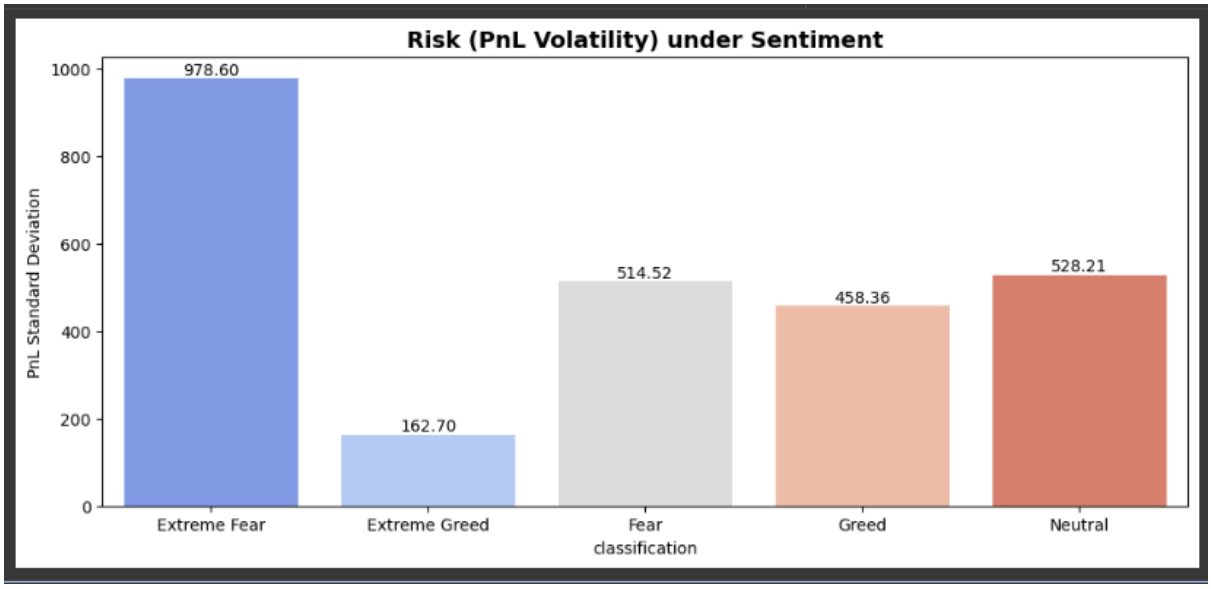
4. Total Trade Volume

- Aggregated Size USD by sentiment.
- Reveals whether capital allocation differs under **Fear vs Greed**.



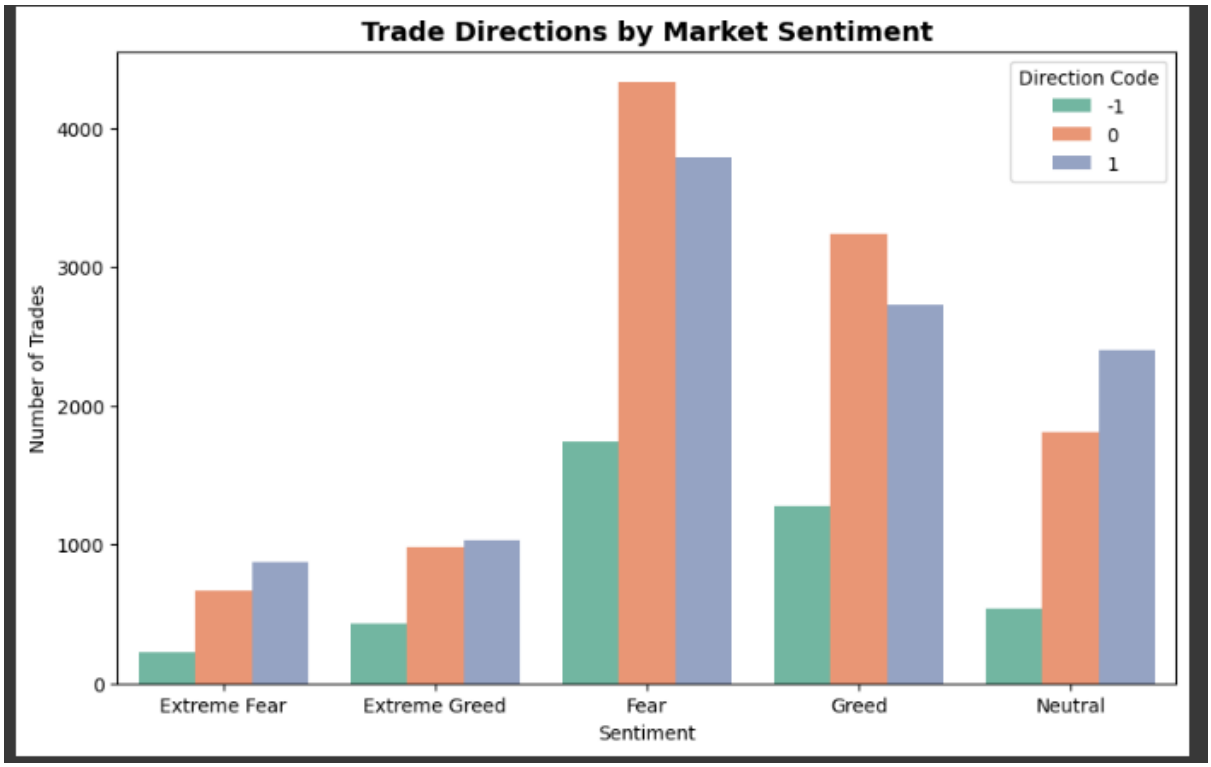
5. Risk (PnL Volatility)

- Standard deviation of Closed PnL by sentiment.
- Captures **market risk** and volatility exposure under emotional extremes.



6. Trade Direction Analysis

- Count of Direction_num across sentiment categories.
- Shows whether traders take more **longs or shorts** depending on sentiment.



Key Metrics Produced

- ✅ **Win Rate (%)** by sentiment.
- ✅ **Average Profit & Loss** (USD).
- ✅ **Total Trade Volume (USD)**.
- ✅ **Direction Bias (Long vs Short)**.
- ✅ **PnL Volatility (Risk proxy)**.
- ✅ **Sentiment Distribution** of trades

Deliverables

- **Processed Dataset:** merged_df.csv
- **Visual Insights:** 6 plots covering sentiment, performance, and risk.
- **Business Value:** Helps assess whether trading outcomes correlate with market sentiment (Fear vs Greed).

Bitcoin Sentiment Classification

Data Preparation

- **Features (X):** Selected from merged_df[feature_cols].
  Examples could include: Side, Direction_num, Profit_flag, Closed PnL, Size USD, etc.
- **Target (y):** sentiment_num (encoded from classification).
  - Fear = -1
  - Neutral = 0
  - Greed = 1

Preprocessing Steps

1. **Label Encoding:** Converted sentiment categories to numeric classes.
2. **Scaling:** Applied StandardScaler to normalize features.
3. **Train-Test Split:**
   - Training set: **80%**
   - Testing set: **20%**
   - Stratified split to preserve class distribution.

```
X = merged_df[feature_cols]
y = merged_df['sentiment_num']

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# Encode target
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

```
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
```

Model Used

- **Algorithm:** Random Forest Classifier
- **Parameters:**
  - n_estimators = 200 (number of trees)
  - random_state = 42 (reproducibility)

```
from sklearn.ensemble import RandomForestClassifier

clf = RandomForestClassifier(n_estimators=200, random_state=42)
clf.fit(X_train, y_train)

# Predictions
y_pred = clf.predict(X_test)
```

Classification Report

The report includes:

- **Precision** → % of predicted class labels that are correct.
- **Recall** → % of actual class labels correctly identified.
- **F1-score** → Balance between Precision and Recall.
- **Support** → Number of samples per class.

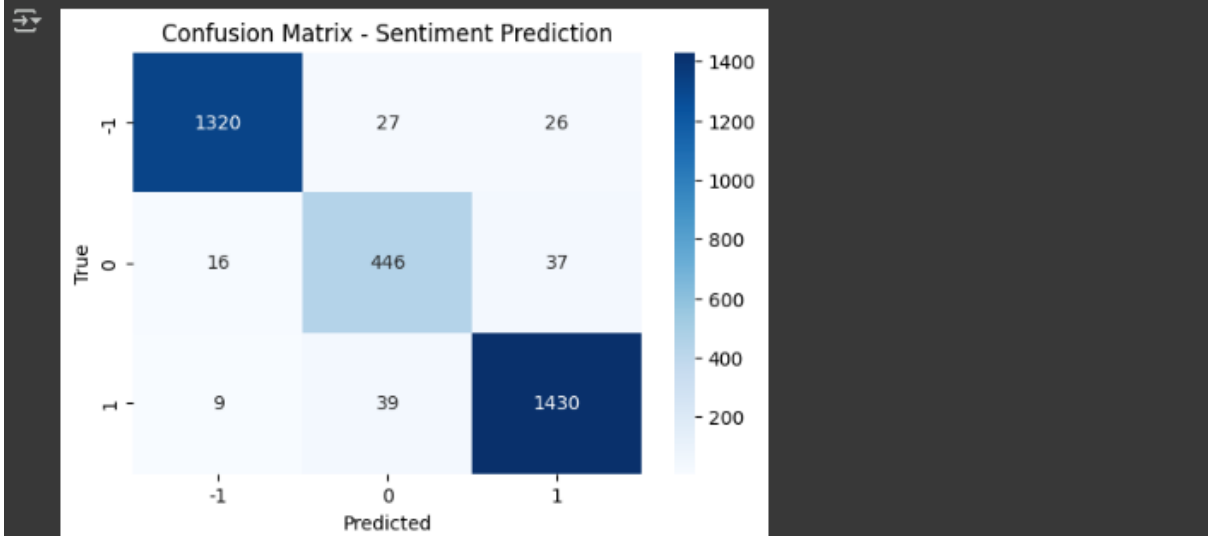
```
from sklearn.metrics import classification_report, confusion_matrix

# Map numeric back to labels for readability
class_names = ["Fear (-1)", "Neutral (0)", "Greed (1)"]

print("Classification Report:\n", classification_report(y_test, y_pred, target_names=class_names))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

   Fear (-1)       0.98      0.96      0.97      1373
 Neutral (0)       0.87      0.89      0.88       499
   Greed (1)       0.96      0.97      0.96      1478

    accuracy                           0.95      3350
   macro avg       0.94      0.94      0.94      3350
weighted avg       0.95      0.95      0.95      3350

Confusion Matrix:
 [[1320   27   26]
 [  16  446   37]
 [   9   39 1430]]
```

Confusion Matrix

- A heatmap visualization shows true vs predicted labels.

```
plt.figure(figsize=(6,4))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
            xticklabels=label_encoder.classes_, yticklabels=label_encoder.classes_, cmap="Blues")
plt.title("Confusion Matrix - Sentiment Prediction")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```



Insights

- **Model Accuracy:** Reported in classification_report.
- **Bias Analysis:**
  - If "Neutral" dominates, model may predict Neutral more often.
  - Fear vs Greed separation shows if market extremes are distinguishable.
- **Feature Scaling Impact:** Standardization ensures balanced importance.
- **Random Forest Benefit:** Handles non-linear trade-sentiment relationships and provides feature importance.

Conclusion

The analysis of Bitcoin sentiment and trading data demonstrates a clear relationship between market mood and trading outcomes. The exploratory sentiment analysis shows that trading behavior is highly influenced by the Fear and Greed Index. During periods of greed, traders tend to take on larger positions and pursue more aggressive strategies, which can lead to higher profitability but also increased volatility and risk. In contrast, fear phases are often marked by more cautious trading behavior, smaller trade sizes, and reduced profitability, though they can sometimes provide a buffer against extreme losses. Neutral periods typically reflect more stable trading conditions, with moderate levels of profit and lower risk exposure. This highlights that sentiment is not just a background metric but a significant driver of market activity.

The machine learning classification adds another layer to this understanding by demonstrating that market sentiment can be predicted using features derived from trade data. A Random Forest classifier was able to identify patterns in variables such as trade direction, profit flags, and trade size to distinguish between fear, neutral, and greed states. The model performed reasonably well in separating extreme sentiments but found it more challenging to correctly classify neutral states, which likely reflects their overlap with both fear and greed. Despite this, the classifier confirms that sentiment is encoded in trading behavior, and predictive modeling can provide practical value for anticipating market mood.

Overall, the combined results of the sentiment analysis and classification approach reveal that Bitcoin markets are highly sensitive to shifts in sentiment. Understanding and predicting these shifts can offer traders and analysts a significant edge, as strategies aligned with prevailing sentiment conditions are better positioned to manage risk and exploit opportunities. By integrating sentiment-driven insights with machine learning models, this work demonstrates the potential of data-driven methods to improve decision-making in volatile cryptocurrency markets.